NAME:J.K.RISHIK

REG.NO:192511052

ASSIGNMENT 2

TASK 1

PROGRAM 1

```
C:\Users\surit\Desktop\C++\ASSIGNMENT 2\TASK 1\program1.exe
1 12 13
1 100
16 13 6

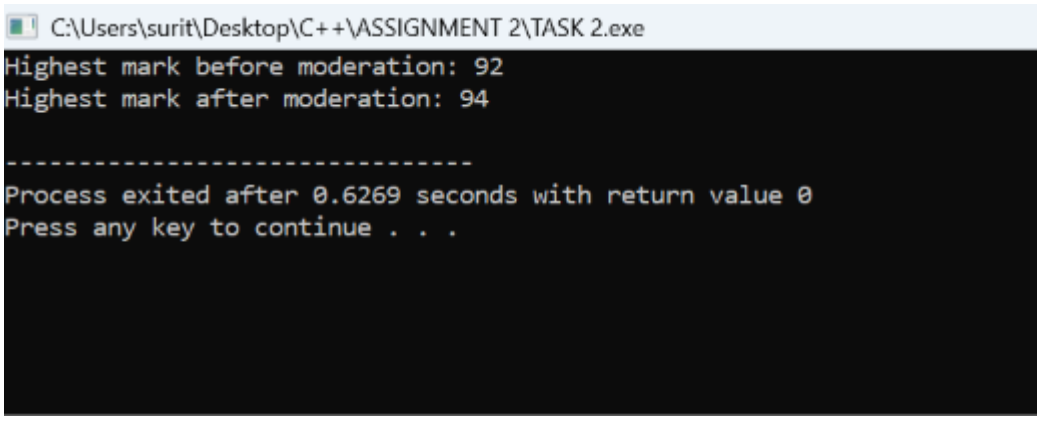
--------------------------------
Process exited after 0.5296 seconds with return value 0
Press any key to continue . . .
```

PROGRAM 2

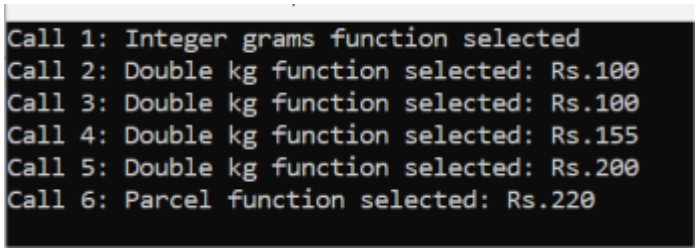
```
C:\Users\surit\Desktop\C++\ASSIGNMENT 2\TASK 1\program2.exe
after call by value : 45000
after call by reference: 40500

--------------------------------
Process exited after 0.5579 seconds with return value 0
Press any key to continue . . .
```

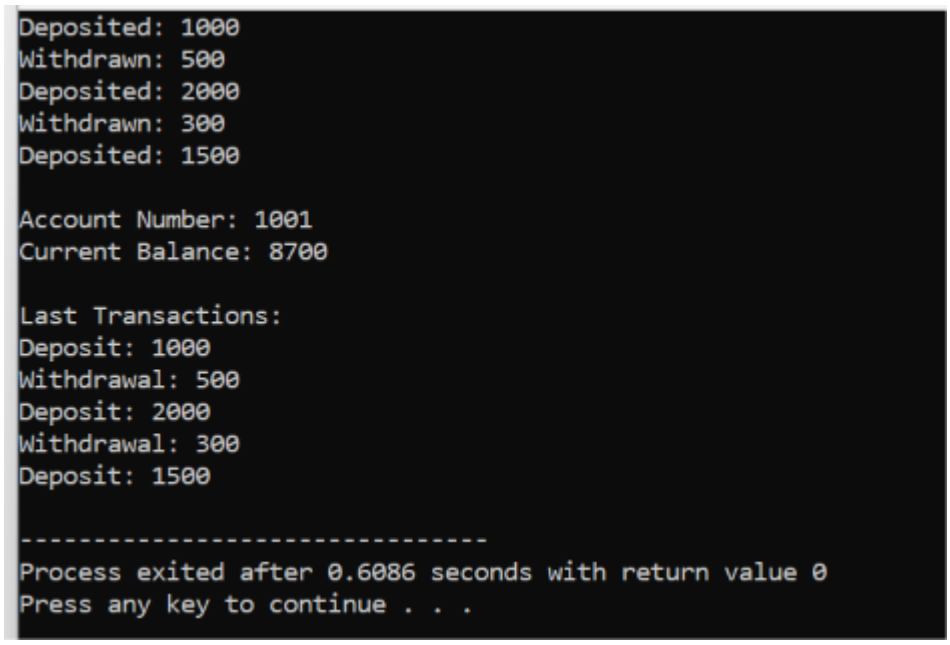
TASK 2



TASK 3



TASK 4



TASK 5

PROGRAM 5.2

```
101 103 104 4

--------------------------------
Process exited after 0.5617 seconds with return value 0
Press any key to continue . . .
```

PROGRAM 5.2

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3

--------------------------------
Process exited after 0.5689 seconds with return value 0
Press any key to continue . . .
```

TASK 6

```
205 false

--------------------------------
Process exited after 0.5333 seconds with return value 0
Press any key to continue . . .
```

TASK 7

A. Diagnose the Limitation – 2 Marks

The original SavingsAccount program stores the last 10 transactions in an array.

When the 11th transaction arrives, the program shifts the existing 9 elements one position to the left to remove the oldest transaction.

Limitation: Repeated shifting wastes time when many transactions are added.

- Original insertion before capacity: O(1)
- Original insertion after capacity: O(10), or O(k) for capacity k.
- For every additional transaction, 9 elements must be shifted when the array is full.

B. Define the Design Objective – 2 Marks

The objective is to redesign transaction storage so that:

- The account stores only the latest 10 transactions.
- The oldest transaction is automatically replaced.
- Each new transaction takes O(1) time, even after the array is full.
- The mini statement displays transactions in chronological order.

C. Design the Revised Solution – 4 Marks

C++ feature used: Circular array with modular arithmetic (%).

Pseudocode:

```
Create array of 10 transactions
Set next = 0
Set count = 0

When a transaction arrives:
   Store it at array[next]
   next = (next + 1) % 10

   If count is less than 10:
      Increase count

To display:
   Find the oldest transaction
   Print count transactions in order
```

Memory: Still uses 10 transaction slots. Only an additional index variable is needed.

Trade-off: The circular array is faster for insertion, but displaying transactions requires calculating their positions using modulo.

D. Predict Before Implementing – 1 Mark

Prediction:

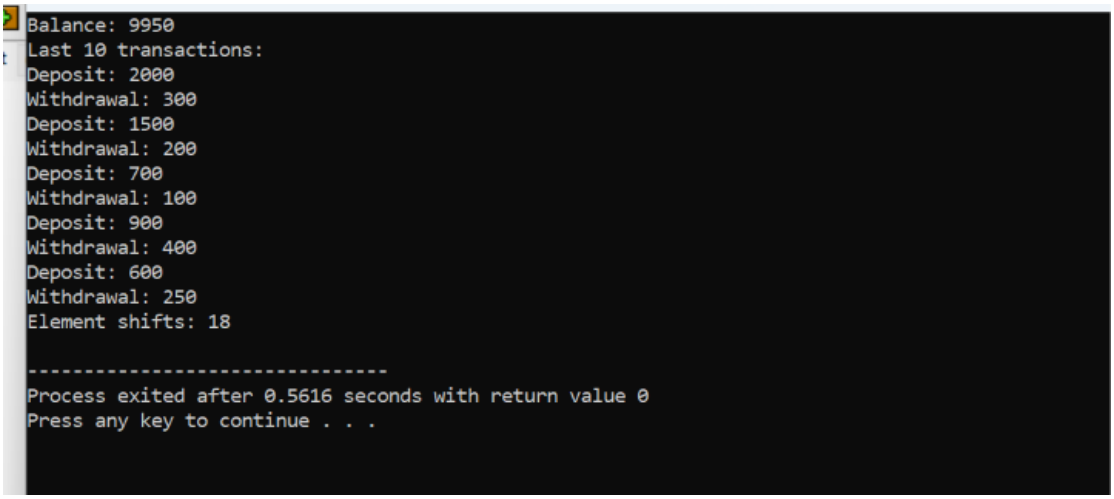
For 12 transactions:

- Original design shifts 9 elements twice = 18 element assignments.
- Redesigned circular array performs 2 direct overwrites and no shifting.
- Both programs should display exactly the same latest 10 transactions.
- Both use O(10) memory

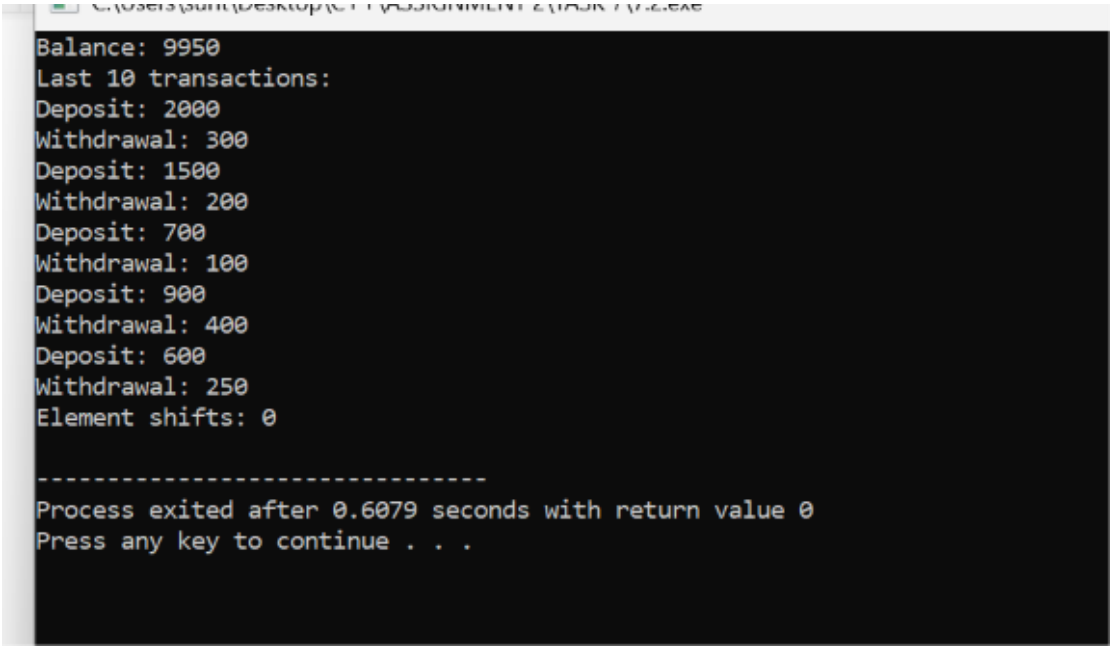
E. Implement and Validate – 2 Marks

Use the same 12 transactions in both programs.

PROGRAM 1



PROGRAM 2



F. Defend the Final Design – 2 Marks

The redesigned circular array achieved the objective.

Advantages:

- Insertion takes O(1) time.
- No shifting of existing transactions.
- Uses the same 10 array positions.
- Displays the latest 10 transactions in correct chronological order.

Disadvantages:

- Requires an additional next index.
- Display logic is slightly more complicated because of modulo arithmetic.

When is the original design preferable?

For a very small number of transactions or when the array is rarely full, the original shifting method is easier to understand and maintain.